



DATA STRUCTURE AND ALGORITHMS

LESSON 01 INTRODUCTION

Lecturer: Ms. SEAK LENG

COMMUNICATION CHANNEL

- Telegram group
- Email: seakleng.itc@gmail.com
- Office: J610

Score Division

- Attendance : 10%
- Quiz : 10%
- Assignment : 10%
- Class Participation : 20%
- Midterm (Project): 20%
- Final : 30%

Lecture overview

❑ Overall lectures

1. Introduction to algorithm
2. Basic data types and statements
3. Control structures and Loop
4. Array
5. Function
6. Structure

**Basic C++
in previous class**

7. Recursive
8. Pointers
9. Linked Lists
10. Stacks and Queues
11. Sorting algorithms
12. Trees

**C++
This course**

Data Structure

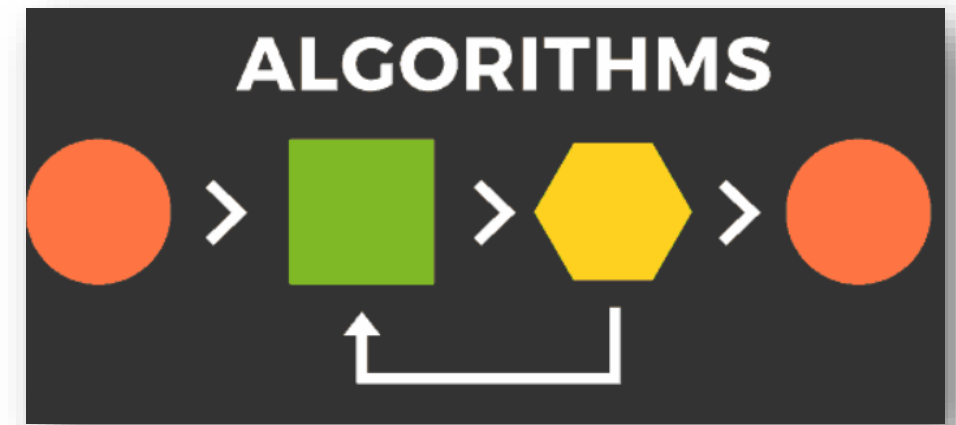
❑ What is a data structure?

- A data structure is a way of organizing and storing data so that it can be accessed and modified efficiently.
- Common types of data structures: Array, Linked List, Stack, Queue, Hash Table, Tree, Graph.
- Data structures are important because
 - They help manage large amounts of data efficiently,
 - They make programs more efficient and scalable
 - Choosing the right data structure can greatly improve performance.

Algorithm

❑ What is algorithm?

- An algorithm is a **step-by-step procedure** describing how to solve a problem / reach a goal
- It may take some *input*, *process the input* to achieve the goal, and finally display the *output/result* if needed
- Example:
 - How to make a fruit salad
 - How to validate an email address
 - How to add two numbers from a user's inputs



Algorithm Examples

❑ Example 1: How to make a “Fruit Salad”

**Ingredient
(input)**

You will need:

2 bananas
2 strawberries
2 oranges
2 apple

Procedures

What to do:

1. Wash your hands and clean your cooking area
2. Wash bananas, strawberries, oranges, apples
3. Peel bananas and oranges
4. Cut all the fruits according to recipe
5. Place fruit in a large bowl and mix

**Product
(output)**

Output:

Fruit salad

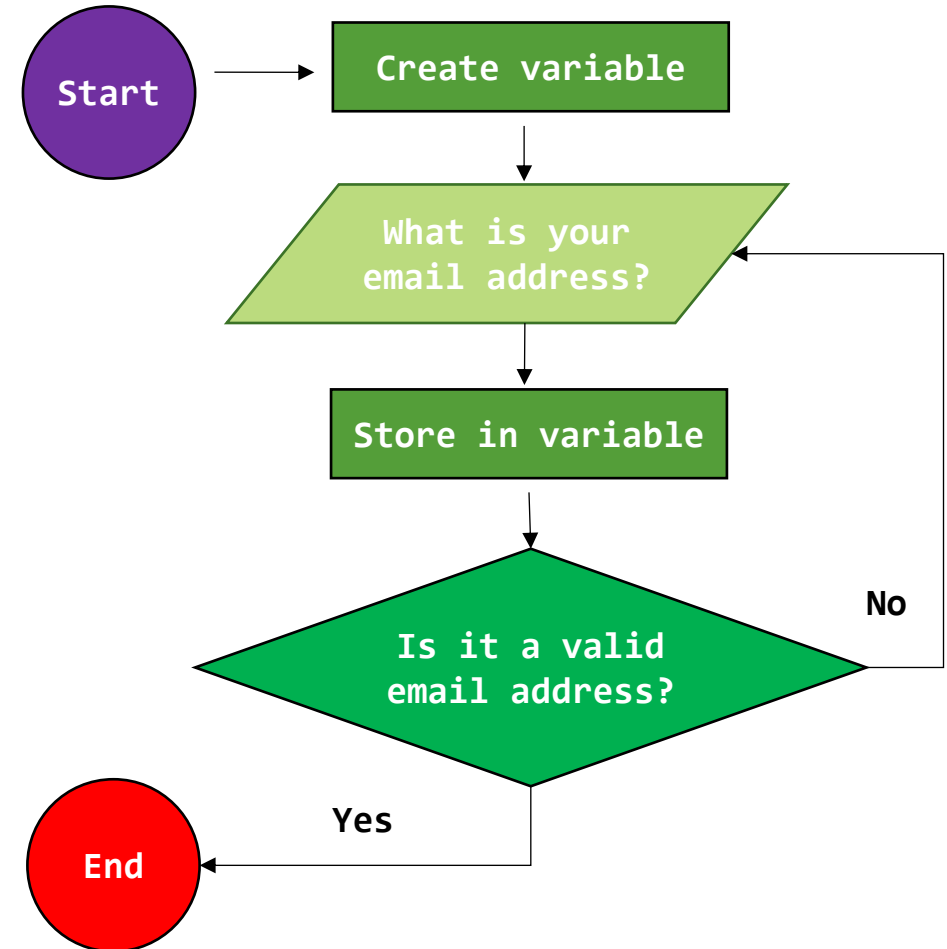
Algorithm Examples

❑ Example 2: How to validate an input email address

START

- Declare variable *address* (text)
- Ask user to input address
- Do operation: check address if valid
 - If not valid, user input again
 - If valid, done.

STOP



Algorithm Examples

❑ Example 3: More specific to coding

START

- Declare variable a, b, c (Number)
- Get values a and b from user
- Do operation $c = a + b$
- Display c

STOP

How to add two numbers

Pseudocode

Var *n1, n2, result* : *Number*

Begin

read(n1, n2)

result $\leftarrow$ *(n1+n2)*2*

write(result)

End

Multiply the sum of two input numbers with 2
and display the result

Syntax Error and Semantic Error

❑ Syntax error

- A **syntax error** occurs when you write code that does not follow the rules of the programming language.
- It is like grammar mistakes in English.
- The compiler will catch these errors and stop your program from compiling
- Example of syntax error:
 - `int x = "a";`

❑ Semantic error

- A **semantic error** occurs when the code compiles successfully, but the logic is incorrect.
- The program runs, but it does not do what you intended.
- It is like using correct grammar but saying something that doesn't make sense.
- Example of semantic error:
 - `int sum = 5+9;`

Remarks

❑ Algorithm and Computer Programming Language

- An algorithm is not computer code
 - It's just written in simple English or whatever the programmer speaks
 - But it has a *start*, a *middle* and an *end*
- Converting algorithm to a specific **computer programming language** is needed to make computer understand and solve a given problem
 - C, **C++**, C#, Java, PHP, JavaScript etc.

Review Basic C++ Programming

❑ What is C++?

- C++ is a high-performance, general-purpose programming language developed by Bjarne Stroustrup in 1983 as an extension of C.
- It supports multiple programming paradigms, including procedural, object-oriented, and generic programming.

Review Basic C++ Programming

❑ Procedural Programming

- **Procedural programming:** Writing code in a sequence of steps or instructions, like following a recipe

Think of it as doing one step after another, like in a cooking recipe.

```
#include <iostream>
using namespace std;

void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    swap(x, y);
    cout << x << " " << y;
    return 0;
}
```

Review Basic C++ Programming

❑ Object-Oriented Programming (OOP)

- **Object-Oriented programming (OOP):** Organizing code using objects (like real-world things) that have properties and actions.

Code is organized into objects — like describing a car with its properties and what it can do.

```
#include <iostream>
using namespace std;

class Car {
public:
    string brand;

    void drive() {
        cout << brand << " is driving!" << endl;
    }
};

int main() {
    Car myCar;
    myCar.brand = "Toyota";
    myCar.drive();
    return 0;
}
```

Review Basic C++ Programming

❑ Generic programming

- **Generic programming:** Writing code that works with any data type, using templates (like writing one recipe that works for any ingredient)

Write code that works for any type — like making a reusable box that can hold anything

```
#include <iostream>
using namespace std;

template <typename T>
T add(T a, T b) {
    return a + b;
}

int main() {
    cout << add(3, 4) << endl;
    cout << add(2.5, 3.5) << endl;
    return 0;
}
```

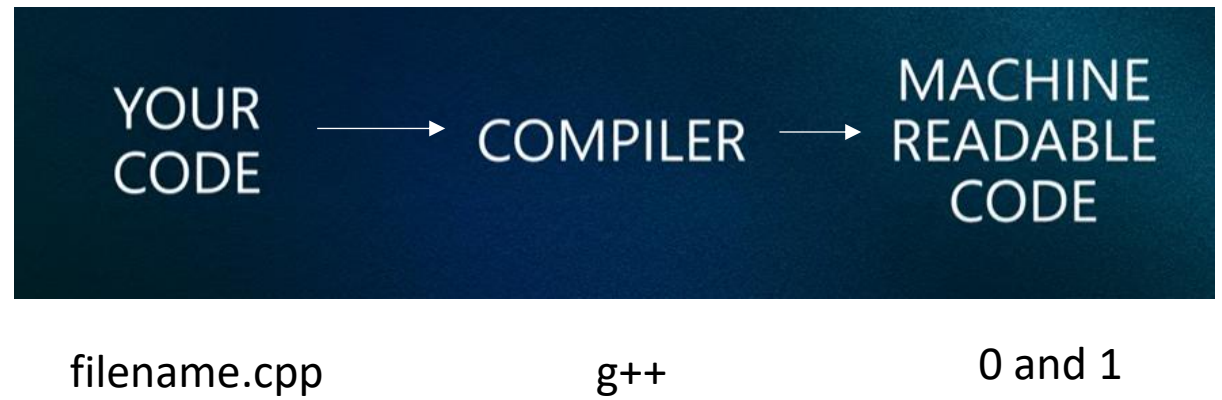
Set up Environment for C++ Programming

❑ C++ Compiler

C++ is a compiled language. To get a C++ program to run, it must be translated from human-readable form source code to something a machine can understand. That translation is done by a program called a compiler.

C++ Compiler:

- MinGW for Windows
- CLANG for Mac OS
- GCC/G++ for Linux



Compiling C++ Language

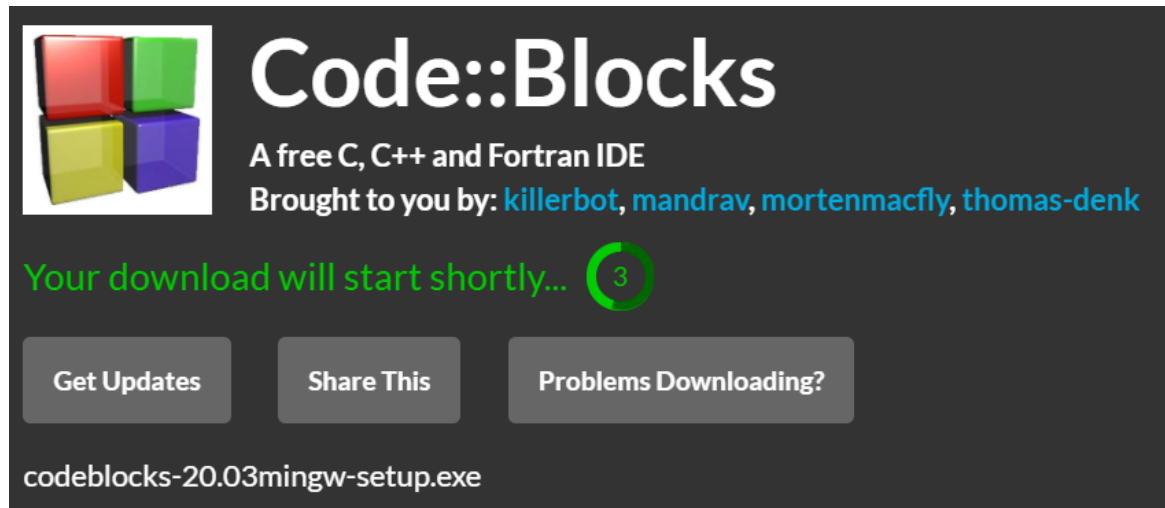
❑ Compiling C++ Language

- Source Code: filename.cpp
- How to compile and execute code with command prompt or terminal
 - `g++ filename.cpp -o filename && ./filename`
- Executable file
 - filename.exe

C++ in CodeBlock IDE

❑ C++ in CodeBlock IDE

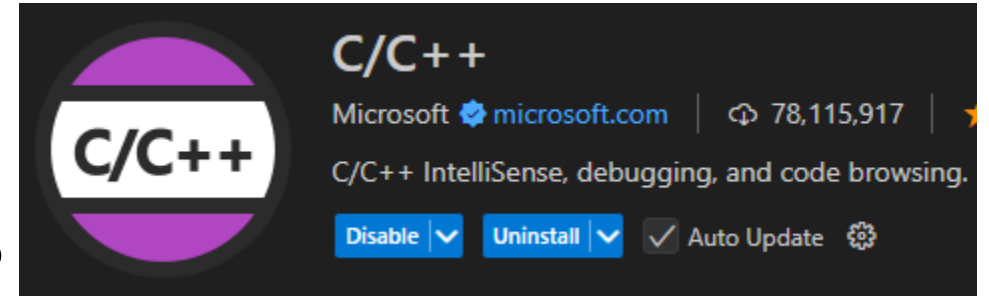
- You can use any IDE
- Install Code::Blocks file that includes the GCC/G++ compiler from MinGW-W64 project for Windows
- [Download codeblocks-20.03mingw-setup.exe \(Code::Blocks\)](#)



C++ in VS Code IDE

❑ C++ in VS Code IDE

- In VS Code, install Extension
 - C/C++ by Microsoft
- In VS Code, press Ctrl + Shift + P
 - Type “open”, press Enter
 - Get Started with C++ Development
 - Set up your C++ Environment
 - Install a compiler: C:\msys64\ucrt64\bin
 - Select Use g++ for Windows or Use clang++ for MacOS
- Add a debug configuration -> Select a compiler
- On source file .cpp, click Run C/C++ File



C++ Basic Syntax

□ Basic Syntax

- Every C++ program starts with a `main()` function. The `#include <iostream>` directive allows input and output operations.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(){
6      |
7      cout << "Hello CPP" << endl;
8      return 0;
9      |
10 }
```

- `#include <iostream>`: Allows input (`cin`) and output (`cout`).
- `using namespace std;`: Simplifies code by avoiding `std::cout`.
- `int main() {}`: Main function, where execution starts.
- `cout << "Hello, World!";`: Outputs text.
- `endl` : move to the next line

C ++Variables and Data Types

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int age = 25; // Integer type
6      double price = 99.99; // Floating-point type
7      char grade = 'A'; // Character type
8      bool isPassed = true; // Boolean type (true/false)
9
10     cout << "Age: " << age << endl;
11     cout << "Price: " << price << endl;
12     cout << "Grade: " << grade << endl;
13     cout << "Passed: " << isPassed << endl;
14
15     return 0;
16 }
```

Operator

❑ Types of Operators:

- Arithmetic: +, -, *, /, %
- Comparison: ==, !=, <, >, <=, >=
- Logical: &&, ||, !
- Bitwise: &, |, ^, ~, <<, >>
- Assignment: =, +=, -=, *=, /=
- Increment/Decrement: ++, --
- Ternary: condition ? true_value : false_value

Operator

❑ Example

- Example of Arithmetic operators:

```
int a = 10, b = 3;
const double PI = 3.14159; // Constant value
int radius = 5;
double area = PI * radius * radius;

cout << "Addition: " << a + b << endl;
cout << "Subtraction: " << a - b << endl;
cout << "Multiplication: " << a * b << endl;
cout << "Division: " << a / b << endl; // Integer division
cout << "Modulus: " << a % b << endl; // Remainder
cout << "Area of circle: " << area << endl;
```

Use **const** to define fixed values that cannot be changed.

Output:

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3
Modulus: 1
Area of circle: 78.5397
```

Control Flow Statements

❑ Control Flow

- Conditional Statements: if, if-else, switch-case
- Loops:
 - for, while, do-while
 - break, continue
- Goto Statement: goto label;

Conditional Statements

❑ if, if-else

```
int num;
cout << "Enter a number: ";
cin >> num; // User input

if (num > 0) {
    cout << "Positive number" << endl;
} else if (num < 0) {
    cout << "Negative number" << endl;
} else {
    cout << "Zero" << endl;
}
```

Conditional Statements

❑ Switch-case

```
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5
6      int opt;
7      cout << "Enter number (1-3):";
8      cin >> opt;
9      switch(opt){
10         case 1:
11             cout << "You enter option 1" << endl;
12             break;
13         case 2:
14             cout << "You enter option 2" << endl;
15             break;
16         case 3:
17             cout << "You enter option 3" << endl;
18             break;
19         default:
20             cout << "Invalid option. Please enter number (1-3)" << endl;
21             break;
22     }
23 }
```

Conditional Statements

❑ Ternary condition

- Ternary: condition ? true_value : false_value

```
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5
6      int b = 10;
7      string result = (b>10) ? "bigger\n" : "smaller or equal\n";
8      cout << result << endl;
9
10     return 0;
11 }
12
```

Loops

❑ For Loop, While Loop

```
//FOR LOOP
for (int i = 1; i <= 5; i++) {
    cout << "Iteration: " << i << endl;
}

//WHILE LOOP
int i = 1;
while (i <= 5) {
    cout << "While Loop: " << i << endl;
    i++;
}
```

Loops

❑ DO-WHILE LOOP

```
//DO-WHILE LOOP
int i = 1;
do {
    cout << "Do-While Loop: " << i << endl;
    i++;
} while (i <= 5);
```

Loops

❑ Nested loop

```
int i;
for (i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        cout << "(" << i << "," << j << ") ";
    }
    cout << endl;
}
```

break

□ break

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      for (int i = 1; i <= 10; i++) {
6          if (i == 5) {
7              cout << "Breaking at " << i << endl;
8              break; // Exit the loop when i is 5
9          }
10         cout << i << " ";
11     }
12     cout << "Loop exited." << endl;
13     return 0;
14 }
```

continue

❑ continue

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      for (int i = 1; i <= 10; i++) {
6          if (i == 5) {
7              cout << "Skipping " << i << endl;
8              continue; // Skip printing 5 and continue the loop
9          }
10         cout << i << " ";
11     }
12     cout << "Loop finished." << endl;
13     return 0;
14 }
```

Goto

❑ goto label;

```
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5
6      int n;
7      cout << "Enter a number: " << endl;
8      cin >> n;
9
10     if(n<0){
11         goto error;
12     }
13
14     cout << "You entered a number " << n << endl;
15     return 0;
16
17     error:
18         cout << "Error: negative number is not allowed\n" ;
19         return 1;
20 }
```

Array

❑ 1D array

```
#include <iostream>
using namespace std;

int main() {
    int numbers[5] = {10, 20, 30, 40, 50}; // Declaring and initializing an array

    // Accessing array elements using a loop
    for (int i = 0; i < 5; i++) {
        cout << "Element at index " << i << ": " << numbers[i] << endl;
    }

    return 0;
}
```

Array

❑ 2D array

```
#include <iostream>
using namespace std;

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Functions

❑ Function Definition

```
#include <iostream>
using namespace std;

int add(int a, int b) { // Function definition
    return a + b;
}

int main() {
    int sum = add(5, 10); // Function call
    cout << "Sum: " << sum << endl;
    return 0;
}
```

Functions

❑ Function Declaration

```
1  #include <iostream>
2
3  using namespace std;
4
5  int add(int a, int b); // Function Declaration
6
7  int main(){
8
9      cout << "Result is " << add(50,45) << endl;
10
11     return 0;
12 }
13
14 int add(int a, int b){
15     return a+b;
16 }
```

Function declaration

Function call

Function definition

Functions

❑ Function Overloading

Function overloading

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}

double add(double a, double b) {
    return a + b;
}

int main() {
    cout << "Sum of 5 and 10: " << add(5, 10) << endl;
    cout << "Sum of 2, 3, and 4: " << add(2, 3, 4) << endl;
    cout << "Sum of 3.5 and 2.2: " << add(3.5, 2.2) << endl;
    return 0;
}
```

Structure

❑ Structure

```
#include <iostream>
using namespace std;

struct Person{
    string name;
    int age;
    char gender;
};

int main(){
    Person p1;
    p1.name = "sok";
    p1.age = 10;
    p1.gender = 'M';

    cout << p1.name << " " << p1.age << " " << p1.gender << endl;
}
```

Practice

- What is the output of the program?

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 20;
    int average = a - b / 2;
    cout << "Average is " << average;
    return 0;
}
```

Practice

- What is the output of the program?

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {10, 20, 30, 40};
    cout << arr[2];
    return 0;
}
```

Practice

- What is the output of the program?

```
#include <iostream>
using namespace std;

int main() {
    int x = 5
    cout << "Value of x is " << x;
    return 0;
}
```

Practice

- What is the output of the program?

```
#include <iostream>
using namespace std;

int main() {
    int x = 5
    cout << "Value of x is " << x;
    return 0;
}
```

Practice

- What is the output of the program?

```
#include <iostream>
using namespace std;

void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    swap(x, y);
    cout << x << " " << y;
    return 0;
}
```

Practice

1. Write a C++ program to add all the numbers input by a user until the user inputs zero. Notice that a number is input one by one until user inputs zero, then display the summation and stop the program.
2. Write a C++ program to check whether an input character is a vowel or a consonant. Hint: Use ASCII code to test condition.
 - Uppercase letters from: 65 to 90, lowercase letters from 97 to 122
 - Vowel and its ASCII code: a = 97, e = 101, i = 105, o = 111, u = 117

Practice

3. Write a C++ program to create a new data structure for storing info of book (book ISBN, title published year, author and price (\$)).Then do each of the following

- Create an array that can store 7 books' info.
- Create a function to display a book info based on ISBN. This function takes a parameter which is an ISBN of a book.
 - `void displayBookByISBN(Book books[], int size, string isbn){.... }`
- Create a function to display information of all books.
 - `void displayAllBooks(Book books[], int size) { ... }`

Practice

4. Write a program to reverse the contents of a 1D array.

Example:

- Input : 1 2 3 4 5 6
- Output : 6 5 4 3 2 1

5. Write a function to transpose a **3 x 3** matrix.

- The function takes in two 2-D arrays as parameters, where the first array represents the original matrix, and in the second array, you need to store the transposed matrix.

Input	Output
1 2 3 4 5 6 7 8 9	1 4 7 2 5 8 3 6 9

Practice

6. Write a C program to allows the user to be able to perform 5 operations below:

a-Tell a user whether a number is Prime number or not using function

b-Display all odd numbers from 0 to n and display all even numbers from 0 to m using the function, where n and m are the parameters of the function.

c-Display all prime numbers between n and m using function, where n and m are the parameters of the function.

d-Compute summation of a suite: $1^2 + 2^2 + 3^2 + \dots + n^2$, where n is a user input.

e-Exit program. The program tells the user the number of times that the user has tried the menu. Then, close the program.

Q & A